UNIT 3 (Operating Systems)

1)What is Critical Section problem?

Critical Section:

-A portion of code in which a process or thread accesses shared resources (like shared variables or data).

-When a process is in its critical section, no other process should be allowed to enter its own critical section for the same resource to avoid conflicts.

-In a code called as Critical section, only one process executes at a time. An algorithm needs to be designed which permits at most one process into the critical section at a time, with no deadlock.

-The critical section's problem must obey mutual exclusion, progress and bounded waiting. Any process cannot directly enter.

-First process has to obtain permission for its entry in its critical section. The segment of code which implements this appeal of process is the entry section. When process comes out of the critical section after completing its execution ,there it has to execute exit section.

-Any solution to the critical-section problem must satisfy the following three necessary conditions:

 Mutual Exclusion:

Only one process can execute in the critical section at any given time.

This property prevents data inconsistency by ensuring only one thread can modify the shared resource simultaneously.

 Progress:

If no process is in the critical section, then any process that wants to enter should be able to proceed without unnecessary delays.

This ensures that no process is unnecessarily blocked from entering its critical section.

 Bounded **Waiting**:

There should be a limit on the number of times other processes can enter their critical sections after a process has requested access.

Bounded waiting prevents starvation, meaning no process should have to wait indefinitely to enter its critical section.

Solutions:

-Semaphore and monitor are the solutions to achieve mutual exclusion. A synchronization variable taking positive integer values is called a Semaphore. Binary Semaphore only has two values 0 or 1. Semaphore offers the solution to the critical problem. If semaphore variable takes value greater than 1 then it is called counting semaphore.

-Like semaphore, a monitor also solves critical section problem. It is a software component which contains one or more procedures, an initialization sequence and local data .Following are the components of monitors:

i)Shared Data Declaration ii)Shared data initialization

iii)Operation on Shared data iv)Synchronization Statement

-**Peterson's Solution**: A software-based approach for two processes. It uses two flags and a turn variable to control access.

When a process wants to enter its critical section, it sets its flag to indicate interest, checks the

turn variable, and waits if the other process is in the critical section.

---

2) i) Mutual Exclusion ii) Synchronization iii) Race condition.

i) Mutual Exclusion: Mutual Exclusion is a property in concurrent programming that ensures only one process or thread can enter the **critical section** at a time to access shared resources. This prevents multiple processes from concurrently modifying shared data, which could lead to inconsistencies.

Example: Consider two processes (A and B) trying to update a shared variable x:

-Process A reads the value of x, increments it, and writes the updated value.

-Simultaneously, Process B reads the same value of x, increments it, and writes the new value.

Without **mutual exclusion**, both processes could read x at the same time and increment the old value, causing an incorrect result.

Using a **mutex** (mutual exclusion lock) solves this:

-Process A locks the mutex, enters the critical section, updates x, and then unlocks the mutex.

-Process B cannot enter the critical section while A holds the mutex. It waits until A releases the lock before proceeding.

This ensures that only one process can access the shared variable x at any time, preventing data inconsistency.

ii) Synchronization: Synchronization refers to the coordination of processes or threads to ensure the correct sequence of execution, especially when accessing shared resources. Synchronization techniques are used to enforce mutual exclusion, ordering, and communication between threads to avoid conflicts like race conditions and ensure consistency.

Example: Imagine a scenario where two threads need to execute in a specific order:

-Thread A: Prepares data.

-Thread B: Uses the prepared data to perform further operations

Without synchronization, Thread B might start before Thread A finishes preparing the data, leading to incorrect results. A synchronization mechanism like a semaphore or condition variable ensures that Thread B waits for Thread A to finish.

Pseudocode:

```
Thread A:
  prepareData();
  signal(threadB);  // Notifies Thread B to proceed

Thread B:
  waitFor(threadA); // Waits for Thread A to finish
  useData();
```

Here, Thread B waits for Thread A to complete, ensuring proper synchronization.

iii) **Race Condition**: A race condition occurs when two or more processes or threads try to access and modify shared data simultaneously, and the final outcome depends on the timing of their execution. The lack of proper synchronization can lead to unpredictable and incorrect results.

Example:

Suppose two threads, Thread A and Thread B, are both trying to increment a shared counter variable count:

- Thread A reads the current value of count as 5.
- Thread B also reads the value of count as 5.
- Both threads increment their local copies of count to 6.
- Both threads write back the value 6 to the shared variable count.

The correct result should have been 7 (because both threads should increment count from 5 to 6, and then to 7). However, due to the race condition, the final value of count is incorrectly set to 6.

To fix the race condition, a mutual exclusion mechanism like a mutex is required:
lock(mutex);
count = count + 1;
unlock(mutex);
With this, Thread A and Thread B cannot access count at the same time, avoiding the race condition.

---

3) Readers-Writers Problem

The **Readers-Writers Problem** is a classic synchronization problem that illustrates issues with multiple processes accessing a shared resource, specifically in scenarios where processes can either **read** or **write** to a shared resource.
The Readers-Writers Problem involves two types of processes.

1. **Readers**: Processes that only read data from a shared resource and do not alter it.
2. **Writers**: Processes that can modify or write data to the shared resource

The goal is to ensure **mutual exclusion** and **consistency** when both readers and writers need access to a shared resource, but with the following requirements:

1. Multiple readers should be able to read simultaneously without causing data inconsistency.
2. Only one writer should be able to write at a time, as concurrent modifications could lead to inconsistencies.
3. No reader should be reading while a writer is writing to prevent readers from seeing inconsistent or partial data.

Types of Readers-Writers Problems

1. **First Readers-Writers Problem (Readers Priority)**:
- This variation gives priority to readers, allowing them to access the resource when there are no writers currently writing.
- If there is a writer waiting to access the resource, new readers can still read, which can lead to a writer **starvation** issue since a constant flow of readers can prevent a writer from ever acquiring the resource.

2. **Second Readers-Writers Problem (Writers Priority)**:
- This version prioritizes writers over readers. When a writer wants access, readers must wait until the writer has finished.
- This approach prevents writer starvation but may cause **reader starvation**, as a continuous stream of writers could delay readers indefinitely.

3. **Third Readers-Writers Problem (Fair Solution)**:
A balanced approach that avoids starvation for both readers and writers. It ensures that when both readers and writers are waiting, they get access in the order they arrived.

Solution Explanation for the First Readers-Writers Problem (Readers Priority)

To handle the Readers-Writers Problem, semaphores or mutexes are commonly used. Below is a solution to the first version of the problem, which gives priority to readers.

Variables and Semaphores Used

1. **read_count**: A counter to keep track of the number of active readers.
2. **mutex**: A semaphore or lock to manage updates to read_count.
3. **rw_mutex**: A semaphore to enforce mutual exclusion for writers, ensuring only one writer can

write or that no readers are present when a writer writes.

Process for Readers

1. Reader Entry:

-A reader first locks mutex to safely update read_count.

-If it is the first reader (read_count == 1), it locks rw_mutex, preventing writers from writing.

-The reader then unlocks mutex to allow other readers to access the resource.

2. Reader Exit:

-After reading, the reader locks mutex and decrements read_count.

-If it is the last reader (read_count == 0), it unlocks rw_mutex, allowing writers access.

-The reader then unlocks mutex.

Process for Writers

1.Writer Entry:

- A writer waits for rw_mutex to be unlocked. When it is unlocked, the writer locks rw_mutex to enter its critical section and start writing.

2. Writer Exit:

- After writing, the writer unlocks rw_mutex, allowing either readers or other writers to access the resource.

Pseudo Code:

```
// Global variables
int read_count = 0;
Semaphore mutex = 1;
Semaphore rw_mutex = 1;

// Reader process
Reader() {
  wait(mutex);
  read_count++;
  if (read_count == 1) {
     wait(rw_mutex); // First reader locks rw_mutex
   }
   signal(mutex);

// Critical section: Reading
wait(mutex);
read_count--;
if (read_count == 0) {
    signal(rw_mutex); // Last reader unlocks rw_mutex
}
signal(mutex);
}

// Writer process
Writer() {
     wait(rw_mutex); // Only one writer at a time
```

```
// Critical section: Writing
    signal(rw_mutex);
}
```

Example Scenario

Imagine a shared document with two types of users:

-----Readers who can view the document without editing.

-----Writers who can make edits.

Multiple readers can view the document simultaneously, but only one writer can edit it at a time, and no reader can view it while it's being edited to ensure data consistency.

---

4) Producer-Consumer Problem (System wake up call)

In the Producer-Consumer Problem, we have two types of processes—producers and consumers—who share access to a buffer with limited storage. The producer's role is to generate data and add it to the buffer, while the consumer's role is to remove data from the buffer and process it. To ensure efficient operation, we need to handle synchronization carefully to avoid buffer overflow (when the producer tries to add to a full buffer) and buffer underflow (when the consumer tries to remove from an empty buffer).

Problem Summary

------Producer: Adds items to the buffer when there is space.

------Consumer: Removes items from the buffer when there are items available.

------Buffer: A shared, bounded (fixed-size) storage structure.

Challenges

------Synchronization: Ensuring that only one process (producer or consumer) accesses the buffer at any time, preventing race conditions.

------Waiting and Signaling: Using system calls like sleep() and wake up() to handle situations when the buffer is full (producer needs to wait) or empty (consumer needs to wait).

Solution with System Wake-Up Calls

------Sleep: If a process (producer or consumer) finds it cannot proceed (due to a full or empty buffer), it calls the sleep() system call. This action moves the process into a waiting state, releasing the CPU so other processes can run.

------Wake Up: When the other process makes the buffer usable again (e.g., the producer frees up a slot by adding an item), it calls the wake up() system call to notify any waiting processes. This reactivates the waiting process, allowing it to resume operation.

Solution Logic with System Calls

Producer Logic:

-----Checks if the buffer is full.

-----If the buffer is full, it calls sleep() to wait for an available slot.

-----When the consumer consumes an item and frees up space, it calls wake up() for the producer.

------The producer then resumes and adds items to the buffer.

Consumer Logic:

-----Checks if the buffer is empty.

------If the buffer is empty, it calls sleep() to wait for an item to be produced.

------When the producer adds an item to the buffer, it calls wake up() for the consumer.

------The consumer resumes, removes the item from the buffer, and processes it.

---

5) Producer-Consumer Problem(Semaphores Solution)

The Producer-Consumer Problem (also known as the Bounded Buffer Problem) is a classic synchronization problem in concurrent programming, which involves two types of processes (or threads): producers and consumers. The goal is to coordinate the producer and consumer processes so that they operate efficiently without conflicts, especially when accessing a shared buffer.

***Problem Overview**

-----Producers generate items and place them into a shared buffer.

-----Consumers remove items from the buffer to process them.

-----The buffer has a finite size (bounded buffer), meaning that it can only hold a certain number of items at any given time.

***Goals:**

-----**Avoid Buffer Overflow**: If the producer tries to add items to a full buffer, it must wait until space is available.

-----**Avoid Buffer Underflow**: If the consumer tries to remove an item from an empty buffer, it must wait until there is at least one item in the buffer.

-----**Ensure Mutual Exclusion**: Only one process (either producer or consumer) should access the buffer at any time to prevent data inconsistencies.

***Key Concepts in the Solution**

---Buffer: A fixed-size data structure (like an array or queue) where the producer places items and from which the consumer retrieves items.

---Synchronization Mechanisms: Semaphores or mutexes are used to manage access to the buffer and coordinate between producers and consumers.

---Semaphores:

--------mutex: Ensures that only one process accesses the buffer at a time (mutual exclusion).

--------empty: Keeps track of the number of empty slots in the buffer.

--------full: Keeps track of the number of filled slots in the buffer.

***Solution Using Semaphores**

Variables and Semaphores Used

buffer[]: The shared buffer array where items are placed by producers and removed by consumers.

mutex: A semaphore initialized to 1, which enforces mutual exclusion, allowing only one process to modify the buffer at a time.

empty: A semaphore initialized to the buffer's maximum capacity. It indicates the number of empty slots available**.**

full: A semaphore initialized to 0, indicating the number of slots currently filled with items.

Producer Process

---Wait for empty: The producer waits until there is at least one empty slot in the buffer (empty > 0).

---Wait for mutex: Locks mutex to gain exclusive access to the buffer.

---Add Item: Produces an item and places it in the buffer.

---Signal mutex: Releases mutex after placing the item.

---Signal full: Increments the full semaphore to indicate a filled slot, allowing the consumer to consume an item.

Consumer Process

---Wait for full: The consumer waits until there is at least one filled slot in the buffer (full > 0).

---Wait for mutex: Locks mutex to gain exclusive access to the buffer.
---Remove Item: Removes an item from the buffer for consumption.
---Signal mutex: Releases mutex after removing the item.
---Signal empty: Increments the empty semaphore to indicate an empty slot, allowing the producer to produce a new item.

Pseudocode

```
Semaphore mutex = 1;      // To ensure mutual exclusion
Semaphore empty = N;      // N is the buffer capacity (empty slots initially equal to N)
Semaphore full = 0;       // No filled slots initially

// Producer process
Producer() {
   while (true) {
      wait(empty);          // Wait until there's an empty slot
      wait(mutex);          // Lock the buffer for exclusive access
      // Critical section: Add item to the buffer
      produceItem();
      addToBuffer(item);
      signal(mutex);        // Release the buffer
      signal(full);         // Increment the filled slot count
      // Perform other tasks or wait to produce the next item
   }
}
// Consumer process
Consumer() {
   while (true) {
      wait(full);           // Wait until there's at least one filled slot
      wait(mutex);          // Lock the buffer for exclusive access
      // Critical section: Remove item from the buffer
      item = removeFromBuffer();
      consumeItem(item);
      signal(mutex);        // Release the buffer
      signal(empty);        // Increment the empty slot count
      // Perform other tasks or wait to consume the next item
   }
}
```

Example Scenario : Consider a real-world example of a printing system:
-----Producer: The process that sends documents to the print queue.
-----Consumer: The printer that takes documents from the queue to print them.
-----Buffer: A limited-sized queue holding documents ready for printing.
When there are too many print jobs (buffer is full), the producer (sending new documents) has to wait. When there are no jobs in the queue (buffer is empty), the printer (consumer) waits until a new job arrives.

---

6) Differentiate between named pipe and unnamed pipe

Feature	Unnamed Pipe (Anonymous Pipe)	Named Pipe (FIFO)
Communication	Between **related processes** (e.g., parent-child) only	Between **unrelated or related processes**
Direction	Typically **unidirectional**	**Bidirectional** (though can be unidirectional as well)
Creation	Created with `pipe()` system call	Created with `mkfifo()` system call or `mkfifo` command in the shell
Existence	Temporary, **exists only while processes are running**	Persistent, **exists as a file** in the filesystem until manually deleted
Identifier	Identified by **file descriptors** returned by `pipe()`	Identified by **pathname** in the filesystem
Use Case	Simple communication between a parent and child process	Communication between unrelated processes or for processes started independently

---

7) Inter-Process Communication (IPC)

1. Shared Memory

Shared Memory is a method where multiple processes can access a common memory area. This allows processes to communicate by reading and writing to shared variables stored in the shared memory segment. Shared memory is one of the fastest IPC mechanisms since it doesn't involve system calls after the initial setup.

Characteristics

-----**Fast Access**: Since data is shared in memory, processes can read and write directly, which is faster than other IPC methods.

-----**Data Sharing**: All processes can access the same memory space, enabling easy sharing of large amounts of data.

----Synchronization Needed: Processes must synchronize access to the shared memory to prevent race conditions (using mutexes, semaphores, etc.).

Example:

Use Case: A data processing application where multiple processes need to access and update a shared dataset, like a collection of sensor readings in a monitoring system.

In such a system, one process might read temperature data from sensors and write it to shared memory, while other processes read from the same shared memory to perform calculations or generate alerts based on the latest readings.

2. Message Passing

Message Passing is a method where processes communicate by sending messages to each other. This mechanism can be used in distributed systems and is often implemented in both synchronous and asynchronous modes.

Characteristics

-----**Decoupled Communication**: Processes do not need to share memory and can be located on different machines.
-----**Message Queue**: Messages are placed in a queue until they are retrieved by the receiving process.
-----**Simplicity**: Message passing abstracts the underlying complexity of managing shared data.
Types of Message Passing
Synchronous: The sending process waits until the receiving process acknowledges receipt of the message.
Asynchronous: The sending process continues without waiting for an acknowledgment, which can improve performance but complicates error handling.
Example:
Use Case: A distributed system where different components (e.g., client-server architecture) need to communicate over a network.
For instance, a web server (the producer) might send messages to a database server (the consumer) to request data or update records. The database server processes these requests and sends back responses in the form of messages, ensuring that both processes can operate independently while still communicating effectively.

3. Pipes
UNIX operating system supports pipe for interprocess communication. It is the circular buffer that permits the processes to communicate on the producer-consumer model. Hence, it is like first-in-first-out queue which is written by one process and read by another.
After creation of pipe a fixed size in bytes is allocated to it.The write request by process is instantly executed if sufficient space is available in buffer. If space is not available then process is blocked.
In the same way, if reading process try to read more bytes than available in buffer then it gets blocked. If bytes are available, then the read request is instantly executed.
The UNIX operating system guarantees mutual exclusion so that single process can access a pipe at a time. Following are the types of pipe:

(i) Named pipe
(ii) Unnamed

The unnamed pipes are shared only by processes which are related to each other. Both processes, related and unrelated can share named pipes.
Example:
Use Case: A command-line interface where the output of one command is used as the input for another command.
For example, in a Unix-like operating system, you can use a pipe to connect the output of the ls command (which lists directory contents) to the grep command (which searches for specific patterns). The command ls | grep "txt" uses a pipe to send the output of ls directly into grep, filtering for files with "txt" in their names.
---

8) Deadlock
Deadlock occurs when a process requests a resource and if the resource is not available at that time, the process enters a wait state. In a multiprogramming environment it may happen with many processes. There is chance that waiting processes will remain in same state and will never again change state. It is because the resources they have requested are held by other waiting processes. When such type of situation occurs then it is called as deadlock.

Necessary and Sufficient Conditions for Deadlock

There are four necessary conditions that must all be present for a deadlock to occur. If any one of these conditions is not met, a deadlock cannot happen. These conditions are often referred to as the Coffman conditions. The four conditions are:

------1. Mutual Exclusion:

----------At least one resource must be held in a non-shareable mode. If one process is using a resource, other processes must be prevented from using it.

----------For example, if a printer is being used by one process, other processes cannot use that printer until it is released.

------2. Hold and Wait:

----------A process holding at least one resource is waiting to acquire additional resources that are currently being held by other processes.

----------For instance, if a process has allocated memory and is waiting for a file to be released by another process, it creates a hold-and-wait condition.

------3. No Preemption:

----------Resources cannot be forcibly taken from a process holding them; they must be voluntarily released by the process.

----------For example, if a process has a lock on a resource, that lock cannot be forcibly removed by the operating system; it can only be released by the process itself.

------4. Circular Wait:

----------There must be a set of processes {P1, P2, ..., Pn} such that P1 is waiting for a resource held by P2, P2 is waiting for a resource held by P3, ..., and Pn is waiting for a resource held by P1.

----------This creates a circular chain of dependencies, preventing any process in the chain from proceeding.

Resource Types and Allocation

Instances of a Resource Type: If there are 5 instances of the same type of resource available in the system, we say there are 5 instances of this resource type. This concept is crucial in understanding how resources are managed and allocated.

Resource Allocation: When a process requests an instance of a resource, if any instance of the resource type is available, the request will be fulfilled. However, if it is not possible to fulfill the request (because all instances are currently in use), it indicates that the resource type classes may not be defined appropriately, or there may be a misallocation of resources.

Resource Usage Sequence

As a rule, a process must follow a sequence when utilizing a resource:

Request: The process requests a resource. If the resource is not immediately available, the requesting process must wait until it can acquire the resource.

Use: Once the resource is allocated, the process uses the resource to accomplish its task.

Release: After completing its work, the process releases the resource, making it available for other processes.

Prevention and Avoidance of Deadlock

Prevention: Altering the system's resource allocation strategies to ensure that at least one of the necessary conditions for deadlock cannot hold.

Avoidance: Using algorithms like the Banker’s Algorithm to ensure that the system will always remain in a safe state, thus avoiding the possibility of deadlock.

Detection and Recovery: Allowing the system to enter a deadlock state but detecting it through methods like resource allocation graphs and recovering by aborting processes or preempting resources.

---

9) Resource Allocation Graph

A Resource Allocation Graph (RAG) is a directed graph used in operating systems to model the allocation of resources to processes and to detect potential deadlocks in a system. It is a visual representation that shows the relationship between processes and resources in a system. It consists of nodes and edges that illustrate the allocation of resources to processes and their requests.

2. Components of RAG

The RAG is defined as follows:

Nodes: Two types of nodes:

Processes (P): Represented by circles, e.g., P1, P2,...

Resources (R): Represented by rectangles, divided into multiple instances.

Edges: Two types of directed edges:

Request Edge (P → R): A directed edge from a process P_i to a resource R_j, indicating that P_i has requested an instance of R_j .

Assignment Edge (R → P): A directed edge from a resource R_j to a process P_i , indicating that an instance of R_j is allocated to P_i.

3. Representation

For example:

P1 → R1 : Process P_1 is requesting resource R_1.

R2 → P2 : Resource R_2 is allocated to process P_2.

A resource R with multiple instances is represented as R_1^m, where m indicates the number of available instances. Each instance can have its assignment edge.

4. Purpose

The RAG helps in:

1. Visualizing resource allocation: Shows which processes hold or request resources.
2. Deadlock detection: Cycles in the graph may indicate a deadlock (depending on resource instances).

5. Deadlock Conditions in RAG

A cycle in the graph:

If every resource in the cycle has exactly one instance, a deadlock is present.

If resources have multiple instances, a cycle does not necessarily imply deadlock.

6. Example

Consider:

P1 requests R1.

P2 is allocated R1.

P3 requests R2, which is held by P1.

Graph Representation:

1. Request Edge: P1 → R1 , P3 → R2 .
2. Assignment Edge: R1 → P2 , R2 → P1 .

A cycle: P1→ R1→ P2→ R2→ P1, indicates a potential deadlock.

7. Characteristics

Dynamic nature: The graph changes as processes request or release resources.

Scalability: Useful for visualizing small systems but may become complex for larger systems.

8. Limitations

Cannot handle processes dynamically sharing resources (e.g., multi-threaded systems).

For systems with multiple resource instances, detecting cycles is not always sufficient for deadlock detection.

9. Use in Deadlock Prevention

Use a safe state detection algorithm (e.g., Banker's Algorithm).

Ensure no circular wait conditions exist by avoiding cycles in RAG.

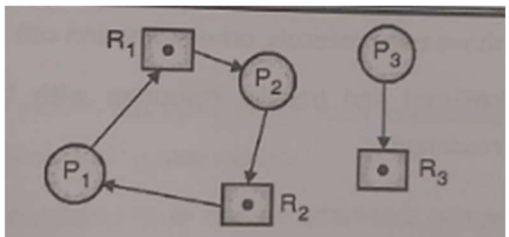

Fig. 3.4.1 : Resource allocation graph